الگوریتم و ساختمان داده

مرتب‌سازی در زمان خطی

از کران پایینِ روش‌های مقایسه‌ای تا Counting Sort و Radix Sort

فارسی و خودآموز | با اثبات و مثال حل‌شده | بر پایهٔ ۲۵ اسلاید

راهنما

مرتب‌سازی در زمان خطی

جزوهٔ خودآموز درس الگوریتم و ساختمان داده

بر پایهٔ فایل Sorting.pdf، شامل ۲۵ اسلاید «Sorting in Linear Time»

این جزوه برای کسی نوشته شده است که مرتب‌سازی درجی، ادغامی و سریع را در حد مقدماتی دیده است. مسیر درس از یک پرسش شروع می‌شود: اگر مرتب‌سازی مقایسه‌ای در بدترین حالت به $\Omega(n \log n)$ زمان نیاز دارد، چطور Counting Sort و Radix Sort گاهی در زمان خطی کار می‌کنند؟ پاسخ در **فرض‌های اضافی دربارهٔ کلیدها** و **روش به‌دست‌آوردن اطلاعات ترتیب** است.

نقشهٔ راه

بخش	اسلایدهای منبع	پرسش اصلی
مرور الگوریتم‌ها و کران پایین	۱ تا ۱۱	چرا مقایسه به‌تنهایی کافی نیست که از $n \log n$ سریع‌تر شویم؟
Counting Sort و پایداری	۱۲ تا ۲۰	چگونه از مقدار کلید برای جای‌گذاری مستقیم استفاده کنیم؟
Radix Sort	۲۱ تا ۲۴	چگونه چند مرتب‌سازی پایدار، عددهای چندرقمی را مرتب می‌کنند؟
جمع‌بندی و خودآزمایی	۲۵ و تمرین‌های تکمیلی	چه زمانی از هر روش استفاده کنیم؟

راهنمای برچسب‌ها. «از اسلاید» یعنی ادعا، مثال یا پرسشی که در فایل اصلی آمده است. «تکمیلی» یعنی توضیح، مثال یا تمرینی که برای خودآموز شدن جزوه افزوده شده است. «اصلاح اسلاید» یعنی متن یا شبه‌کد اسلاید ابهام یا ناسازگاری دارد و اینجا شکل درست آن جداگانه توضیح داده شده است.

قراردادها. n تعداد عناصر، k بزرگ‌ترین کلید در بازهٔ $0..k$، و d تعداد رقم‌هاست. برای Counting Sort، آرایهٔ ورودی $A[1..n]$، خروجی $B[1..n]$، و شمارنده $C[0..k]$ است. اندیس‌ها در مثال‌ها از ۱ شروع می‌شوند، اما کلیدها می‌توانند صفر باشند. نمادهای O ، Ω و Θ به‌ترتیب کران بالا، کران پایین و کران هم‌مرتبهٔ مجانبی را نشان می‌دهند.

مرز محتوا: فایل اصلی فقط این سه محور را درس می‌دهد: کران پایین مرتب‌سازی مقایسه‌ای، Counting Sort و Radix Sort. موضوع‌هایی مثل Bucket Sort یا روش‌های مرتب‌سازی اعشاری در فایل نیستند و در این جزوه هم به‌عنوان محتوای اسلاید معرفی نشده‌اند.

بخش 1

۱. چرا مرتب‌سازی مقایسه‌ای کران پایین دارد؟

۱٫۱ مرور روش‌هایی که تا اینجا دیده‌ایم

از اسلایدهای ۲ تا ۴. مرتب‌سازی درجی در بهترین حالت $O(n)$، و در بدترین و میانگین حالت $O(n^2)$ است. Merge Sort آرایه را دو نیم می‌کند، هر نیمه را بازگشتی مرتب می‌کند و آن‌ها را در زمان خطی ادغام می‌کند؛ بدترین زمان آن $O(n \log n)$ است. Quicksort با انتخاب یک محور، افراز آرایه و فراخوانی بازگشتی کار می‌کند؛ نسخهٔ تصادفی آن زمان موردانتظار $O(n \log n)$ دارد، اما بدترین حالتش $O(n^2)$ است.

روش	بهترین	بدترین	میانگین/موردانتظار	مبنای تشخیص ترتیب
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	مقایسه
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	مقایسه
Randomized Quicksort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$ موردانتظار	مقایسه

شهود. مرتب‌سازی درجی روی دادهٔ ازپیش‌مرتب‌شده فقط همسایه‌ها را می‌سنجد؛ اما اگر عناصر برعکس باشند، هر عنصر باید از چند عنصر عبور کند. Merge Sort حتی در حالت خوب نیز همهٔ سطوح تقسیم و ادغام را انجام می‌دهد. تصادفی‌سازی Quicksort احتمال افرازهای بد را کم می‌کند، نه اینکه بدترین حالت را حذف کند.

۱٫۲ مدل مقایسه و پرسش اسلاید

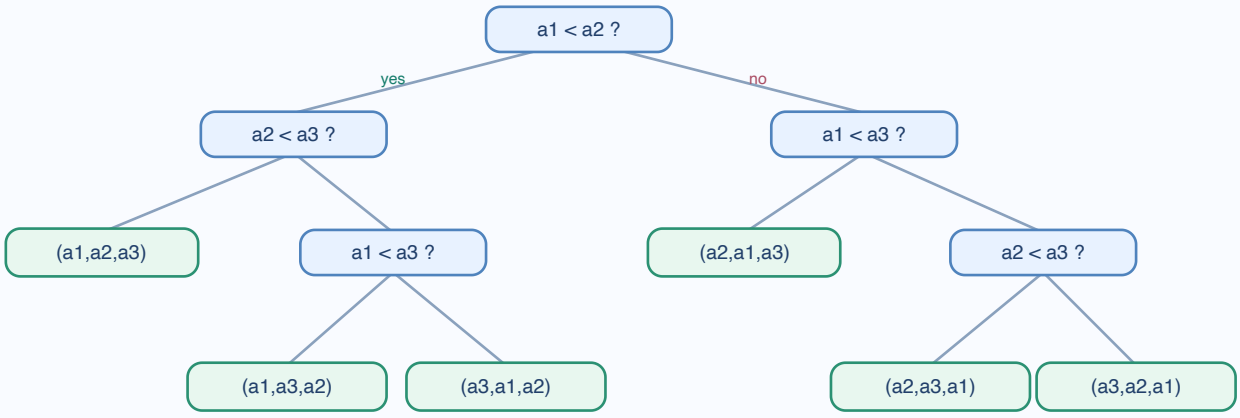
از اسلاید ۵. در «مرتب‌سازی مقایسه‌ای»، الگوریتم فقط از مقایسهٔ دوتایی کلیدها می‌فهمد کدام باید زودتر بیاید. پرسش اسلاید: «چرا دست‌کم به تعداد خطی مقایسه لازم است و بهترین زمان ممکن چیست؟»

پاسخ مرحله‌ای. برای n عنصر متمایز، $n!$ ترتیب اولیهٔ ممکن وجود دارد. یک مقایسهٔ بله/خیر، حداکثر یک بیت از ابهام را رفع می‌کند. برای تمایز $n!$ حالت، شمار مقایسه‌های بدترین مسیر باید دست‌کم $\lceil \log_2(n!) \rceil$ باشد. این مقدار از n هم بزرگ‌تر می‌شود و مرتبه‌اش $n \log n$ است.

> **اصلاح اسلاید ۵:** عبارت «at least n comparisons» برای هر n دقیق نیست؛ مثلاً مرتب‌کردن دو عنصر با یک مقایسه ممکن است. گزارهٔ دقیقِ ساده برای الگوریتم مقایسه‌ای، حداقل $n-1$ مقایسه در بدترین حالت است؛ کران قوی‌تر در ادامه $\lceil \log_2(n!) \rceil$ است. منظور آموزشی اسلاید، نیاز به تعداد **خطی** مقایسه بوده است.

۱٫۳ درخت تصمیم

از اسلایدهای ۶ تا ۸. اجرای یک الگوریتم مقایسه‌ای را می‌توان به صورت درخت دودویی دید. هر گره یک مقایسه است؛ هر یال یکی از دو نتیجهٔ آن و هر برگ یک ترتیب نهایی را مشخص می‌کند. در اسلاید ۷ برای سه عنصر a, b, c، یک الگوریتم فرضی به نام X نمایش داده شده؛ در اسلاید ۸ درخت تصمیم Insertion Sort برای همان سه عنصر آمده است. درخت‌ها شکل یکسانی ندارند، اما هر دو باید بتوانند شش حالت $3! = 6$ را از هم تشخیص دهند.



بازرسم آموزشی درخت اسلاید ۸: هر مسیر از ریشه تا یک برگ، مقایسه‌های لازم برای یک ترتیب ورودی را نشان می‌دهد.

مثال تکمیلی. فرض کن $a < b$ را بپرسیم. اگر درست باشد، هنوز سه ترتیب سازگار داریم: $a < b < c$، $a < c < b$ و $c < a < b$. مقایسهٔ دوم می‌تواند $b < c$ باشد؛ اگر درست شد پاسخ قطعی است، وگرنه برای تفکیک دو حالت باقی‌مانده باید $a < c$ را هم بپرسیم. بنابراین بعضی ورودی‌ها به سه مقایسه نیاز دارند. درخت اسلاید ۷ نیز همین پدیده را با شاخه‌بندی دیگری نشان می‌دهد.

رابطهٔ عمق و هزینه. تعداد مقایسه‌های بدترین ورودی برابر عمق عمیق‌ترین برگ است. اگر همهٔ جایگشت‌ها را با احتمال یکسان در نظر بگیریم، میانگین تعداد مقایسه‌ها برابر میانگین عمق برگ‌های متناظر است. برگ کم‌عمق برای یک ورودی، وجود برگ‌های عمیق‌تر را نفی نمی‌کند.

۱٫۴ اثبات کران پایین، گام‌به‌گام

از اسلایدهای ۹ و ۱۰. اگر عمق درخت h باشد، حداکثر 2^h برگ دارد. چون برای n عنصر متمایز دست‌کم $n!$ برگ لازم است، باید:

$$2^h \geq n! \quad \Longrightarrow \quad h \geq \lceil \log_2(n!) \rceil.$$

اکنون نشان می‌دهیم این عدد $\Omega(n \log n)$ است. در حاصل‌ضرب $n! = 1 \cdot 2 \cdots n$، دست‌کم $n/2$ عاملِ انتهایی هرکدام حداقل $n/2$ هستند؛ برای سادگی n را زوج می‌گیریم. پس:

$$n! \geq \left(\frac{n}{2}\right)^{n/2} \quad \Longrightarrow \quad \log_2(n!) \geq \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \frac{n}{2} \log_2 n - \frac{n}{2}.$$

بنابراین هر الگوریتمی که **تنها** از مقایسه برای کسب اطلاعات ترتیب استفاده کند، در بدترین حالت $\Omega(n \log n)$ مقایسه نیاز دارد. این استدلال دربارهٔ کلیدهای متمایز است؛ همین زیرمجموعهٔ ورودی‌ها برای اثبات کران بدترین حالت کافی است.

از اسلاید ۱۱. Merge Sort با کران بالای $O(n \log n)$ به این کران پایین می‌رسد؛ پس از دید مرتبهٔ مجانبی، یک مرتب‌سازی مقایسه‌ای بهینه است. «بهینه» در اینجا یعنی مرتبهٔ زمانی بهتر در مدل مقایسه‌ای وجود ندارد، نه اینکه در هر دستگاه و برای هر ورودی سریع‌ترین اجرا را داشته باشد.

اصلاح اسلاید ۲۵: Quicksort تصادفی در **امید ریاضی** $O(n \log n)$ و از این نظر هم‌مرتبه با کران پایین است؛ بدترین حالت آن $O(n^2)$ باقی می‌ماند. پس جملهٔ «Merge Sort و Quick Sort هر دو بهینه‌اند» باید با این تفاوت خوانده شود.

مکث و خودآزمایی

1. چرا داشتن $n!$ برگ به معنای نیاز به $n!$ مقایسه نیست؟
2. برای سه عنصر متمایز، کران اطلاعاتی چند مقایسه در بدترین حالت می‌دهد؟

پاسخ کوتاه: ۱) هر مقایسه یک گره در یک مسیر است؛ تعداد برگ‌ها کل حالت‌های ممکن است، و عمق لازم فقط لگاریتم تعداد برگ‌هاست. ۲) $\lceil \log_2 6 \rceil = 3$.

بخش 2

۲. Counting Sort: مرتب‌سازی با شمردن

۲٫۱ فرض مسئله و ایده

از اسلاید ۱۲. فرض کنید n کلید **صحیح** داریم و هر کلید در بازهٔ $0..k$ است. به‌جای مقایسهٔ کلیدها با یکدیگر، تعداد وقوع هر مقدار را می‌شماریم. سپس با جمع تجمعی می‌فهمیم آخرین جای مجاز هر مقدار در خروجی کجاست.

برای مقدار x، عبارت $C[x]$ پس از مرحلهٔ جمع تجمعی یعنی «چند ورودی مقدارشان حداکثر x است؟» بنابراین آخرین x باید در خانهٔ شمارهٔ $C[x]$ خروجی قرار گیرد. بعد از قرار دادن آن، $C[x]$ را یکی کم می‌کنیم تا x بعدی در جای قبلی بنشیند.

تفاوت با مقایسه. الگوریتم از مقدار عددی کلید به‌عنوان اندیس آرایه استفاده می‌کند. این فرض قوی‌تر از «می‌توان دو کلید را مقایسه کرد» است؛ بنابراین کران پایین فصل قبل را نقض نمی‌کند.

> **تکمیلی — دربارهٔ رتبه در اسلاید ۱۲:** اسلاید می‌گوید رتبهٔ x برابر تعداد عناصر کوچک‌تر از آن به‌علاوهٔ یک است. این برای کلیدهای متمایز کافی است. با کلیدهای تکراری، چند عنصر رتبهٔ یکسان می‌گیرند؛ آرایهٔ شمارنده و کاهش تدریجی آن جای آن‌ها را جدا می‌کند.

۲٫۲ سه مرحلهٔ الگوریتم

از اسلایدهای ۱۳ و ۱۷، با اصلاح اندیس‌گذاری:

```
CountingSort(A[1..n], k):
    B[1..n] ← آرایهٔ خروجی
    C[0..k] ← همه صفر

    برای j از 1 تا n:
        C[A[j]] ← C[A[j]] + 1

    برای x از 1 تا k:
        C[x] ← C[x] + C[x-1]

    برای j از n رو به عقب، تا 1:
        x ← A[j]
        B[C[x]] ← A[j]
        C[x] ← C[x] - 1

    B را برگردان
```

در نسخهٔ رکوردی، x را از **کلید** رکورد می‌گیریم ولی خود رکورد کامل را در B می‌گذاریم. اگر فقط مقدارها را ذخیره کنیم، پایداری دیده نمی‌شود؛ معنی پایداری وقتی روشن می‌شود که رکوردها اطلاعات دیگری هم داشته باشند.

اصلاح اسلایدهای ۱۳ و ۱۷: در اسلاید ۱۳ اندازهٔ C به‌اشتباه n توصیف شده، در حالی که برای کلیدهای $0..k$ باید $k+1$ خانه داشته باشد. در شبه‌کد اسلاید ۱۷ مقداردهی از ۱ و جمع تجمعی از ۲ شروع می‌شود، اما مثال همان اسلایدها کلید صفر دارد. شبه‌کد بالا از $C[0]$ شروع می‌کند و با همان مثال سازگار است. در اسلاید ۱۳ یک کروشهٔ دستور جای‌گذاری هم افتاده است.

۲٫۳ مثال کامل اسلایدهای ۱۴ تا ۱۶

از اسلاید. ورودی با اندیس‌های ۱ تا ۸ چنین است:

$$A = [2, 5, 3, 0, 2, 3, 0, 3], \qquad k = 5.$$

در مرحلهٔ شمارش، برای کلیدهای صفر تا پنج داریم:

کلید x	۰	۱	۲	۳	۴	۵
تعداد اولیه	۲	۰	۲	۳	۰	۱
تعدادِ حداکثر x	۲	۲	۴	۷	۷	۸

در اسلاید ۱۴، از انتهای ورودی یعنی $A[8] = 3$ شروع می‌شود: چون $C[3] = 7$، این ۳ در $B[7]$ می‌نشیند و $C[3]$ از ۷ به ۶ می‌رسد. اسلایدهای ۱۵ و ۱۶ دو حرکت بعدی را نشان می‌دهند: $A[7] = 0$ در $B[2]$، سپس $A[6] = 3$ در $B[6]$. جدول زیر کل حرکت‌ها را کامل می‌کند:

اندیس ورودی j	کلید	جای خروجی پیش از کاهش C	وضعیت تازهٔ همان شمارنده
۸	۳	$B[7] = 3$	$C[3] = 6$
۷	۰	$B[2] = 0$	$C[0] = 1$
۶	۳	$B[6] = 3$	$C[3] = 5$
۵	۲	$B[4] = 2$	$C[2] = 3$
۴	۰	$B[1] = 0$	$C[0] = 0$
۳	۳	$B[5] = 3$	$C[3] = 4$
۲	۵	$B[8] = 5$	$C[5] = 7$
۱	۲	$B[3] = 2$	$C[2] = 2$

پس پاسخ نهایی $B = [0, 0, 2, 2, 3, 3, 3, 5]$ است. دقت کن شمارندهٔ تجمعی بعد از جای‌گذاری تغییر می‌کند؛ بنابراین اگر بخواهی همان جدول «تعداد حداکثر x» را دوباره استفاده کنی، باید نسخه‌ای از آن را نگه داری.

۲٫۴ چرا درست است و چرا پایدار است؟

استدلالِ درستی. پس از جمع تجمعی، $C[x]$ تعداد کل عناصر با مقدار $\le x$ است. پس عناصر x باید درست بعد از عناصر کوچک‌تر از x و حداکثر تا خانهٔ $C[x]$ قرار گیرند. هر بار که یک x قرار می‌گیرد، شمارنده یکی کم می‌شود؛ در نتیجه عناصر هم‌مقدار در خانه‌های متفاوت و پشت‌سرهم می‌نشینند. همهٔ خانه‌ها دقیقاً یک بار پر می‌شوند.

از اسلایدهای ۱۸ و ۱۹. مرتب‌سازی پایدار است اگر دو رکورد با کلید مساوی، ترتیب نسبی خود را حفظ کنند. در مثال اسلاید ۱۹، ورودیِ برچسب‌دار چنین است:

```
2₁  7₁  4₁  4₂  2₂  5₁  2₃  6₁
```

و خروجی پایدار باید چنین باشد:

```
2₁  2₂  2₃  4₁  4₂  5₁  6₁  7₁
```

چرا حرکت از **راست به چپ** لازم است؟ اگر $i < j$ و هر دو کلید x دارند، رکورد j زودتر پردازش می‌شود و در آخرین جای آزاد x می‌نشیند. سپس شمارنده کم می‌شود و رکورد i در جای قبلی می‌نشیند؛ پس همچنان پیش از j خواهد بود. اگر همین نسخه را از چپ به راست اجرا کنیم، ترتیب مساوی‌ها برعکس می‌شود.

۲٫۵ زمان، حافظه و محدودیت عملی

از اسلایدهای ۱۷، ۱۸ و ۲۰. صفرکردن و جمع تجمعی روی $k+1$ خانه $O(k)$ زمان می‌برند؛ دو گذر از n عنصر $O(n)$ هستند. پس زمان $\Theta(n+k)$ و حافظهٔ اضافه $\Theta(n+k)$ است. فقط وقتی $k = O(n)$ باشد می‌توان آن را $O(n)$ نسبت به تعداد ورودی‌ها نامید.

پرسش اسلاید ۲۰: «چرا همهٔ عددهای ۳۲بیتی را همیشه با این روش مرتب نمی‌کنیم؟» اگر دامنهٔ کلیدها نزدیک 2^{32} باشد، آرایهٔ C به میلیاردها خانه نیاز دارد. حتی با شمارنده‌های چهاربایتی، فقط خود C حدود ۱۶ گیبی‌بایت حافظه می‌خواهد. بنابراین هزینهٔ k ممکن است بسیار بیشتر از n شود. برای دامنهٔ بزرگ، Radix Sort بخش بعد می‌تواند راه دیگری باشد.

> **تکمیلی — شرط‌های پنهان:** کلید منفی را نمی‌توان مستقیم اندیس C کرد؛ می‌توان با یک جابه‌جایی ثابت، بازهٔ محدود $m..M$ را به $0..M-m$ برد. این تغییر همچنان به **پهنای بازه** وابسته است. همچنین کلیدها باید صحیح یا قابل تبدیل به تعداد محدودی دسته باشند.

پرسش اسلاید ۱۷

زمان اجرا چقدر است؟ **پاسخ:** $\Theta(n+k)$، زیرا سه گذرِ n، k و n انجام می‌شود. $\Theta(n)$ گفتن بدون شرطی روی k درست نیست.

بخش 3

۳. Radix Sort: مرتب‌سازی رقم‌به‌رقم

۳٫۱ چرا از یک رقم شروع می‌کنیم؟

از اسلاید ۲۱. فرض کنید هر کلید d رقم دارد و هر رقم از $0..k$ است. مثلاً برای اعداد ده‌دهی، $k = 9$. Radix Sort نوع **LSD** در اسلاید، از کم‌ارزش‌ترین رقم شروع می‌کند و در هر مرحله همهٔ رکوردها را بر اساس یک رقم، با یک روش **پایدار** مرتب می‌کند. Counting Sort بخش قبل می‌تواند مرتب‌سازی هر مرحله باشد.

شهود تکمیلی. پس از مرحلهٔ یکان، عناصر بر اساس یک رقم آخر مرتب‌اند. پس از مرحلهٔ دهگان، دهگان معیار اصلی می‌شود و پایداری ترتیب یکان را میان عناصری که دهگان برابر دارند نگه می‌دارد. بعد از صدگان، سه رقم آخر به‌ترتیب درست‌اند. پس ترتیب کامل با افزایش تعداد رقم‌های بررسی‌شده ساخته می‌شود.

۳٫۲ شبه‌کد و ناوردای درست‌بودن

از اسلاید ۲۲.

```
RadixSort(A, d):
    برای i از 1 تا d:                 # یعنی کم‌ارزش‌ترین رقم i=1
        A ← StableSortByDigit(A, i) # مثلاً Counting Sort روی همان رقم
    A را برگردان
```

ناوردا: پس از دور i، آرایه بر اساس i رقم کم‌ارزش‌تر مرتب است. پایهٔ استقرا $i = 1$ است. در گام بعد، مرتب‌سازی بر اساس رقم $i + 1$، عناصر را به گروه‌های آن رقم می‌برد؛ در هر گروه، پایداری ترتیب قبلیِ i رقم را نگه می‌دارد. پس کل $i + 1$ رقم درست مرتب می‌شوند.

> **اشتباه رایج:** اگر در هر مرحله از مرتب‌سازی **ناپایدار** استفاده شود، مرحلهٔ دهگان ممکن است ترتیبِ یکان را میان دو عدد با دهگان مساوی به‌هم بزند. صرفاً «مرتب بودن پس از هر گذر» کافی نیست؛ ترتیب نسبی کلیدهای مساویِ همان گذر مهم است.

۳٫۳ مثال کامل اسلاید ۲۳

از اسلاید. هفت عدد سه‌رقمی ورودی:

$$329,\ 457,\ 657,\ 839,\ 436,\ 720,\ 355.$$

مرحله	آرایه پس از مرتب‌سازی پایدار	توضیح
آغاز	329, 457, 657, 839, 436, 720, 355	ترتیب اولیه
یکان	720, 355, 436, 457, 657, 329, 839	رقم‌های یکان: 0،5،6،7،7،9،9
دهگان	720, 329, 436, 839, 355, 457, 657	دهگان: 2،2،3،3،5،5،5؛ ترتیبِ قبلی در هر گروه حفظ شد
صدگان	329, 355, 436, 457, 657, 720, 839	ترتیب نهایی

به جفت 457 و 657 دقت کن: هر دو یکان ۷ و دهگان ۵ دارند. پس در دو گذر اول همان ترتیب اولیه را حفظ می‌کنند. پس از گذر صدگان، چون 4>6 است، 457 پیش از 657 می‌ماند. برای دیدن نقش واقعی پایداری، 329 و 839 را نگاه کن: پس از گذر دهگان در گروه‌های متفاوت‌اند، اما در گذر صدگان هر عدد در گروه صدگان خود قرار می‌گیرد و ترتیب دو رقم قبلی در همان گروه حفظ می‌شود.

مثال تکمیلی برای ضرورت پایداری. اعداد 21 و 22 پس از مرتب‌سازی یکان به صورت 21،22 هستند. هر دو دهگان ۲ دارند. اگر گذر دهگان ناپایدار باشد ممکن است 22،21 خروجی دهد که غلط است؛ روش پایدار ترتیب درست را حفظ می‌کند.

۳٫۴ زمان اجرا و شرایط خطی بودن

از اسلاید ۲۴. اگر هر گذر با Counting Sort روی n عنصر و رقم‌های $0..k$ انجام شود، هزینهٔ هر گذر $\Theta(n+k)$ است. برای d گذر:

$$T(n,d,k) = \Theta\big(d(n+k)\big).$$

اگر d یک ثابت و $k = O(n)$ باشد، این زمان $O(n)$ است. اما اگر تعداد رقم‌ها با ورودی رشد کند یا مبنای انتخاب‌شده بسیار بزرگ باشد، نمی‌توان بی‌قید آن را خطی نامید. حافظهٔ اضافه برای نسخه‌ای که هر گذر را با Counting Sort انجام می‌دهد $O(n+k)$ است و همان فضا را می‌توان بین گذرها دوباره استفاده کرد.

پرسش اسلاید ۲۴: زمان Radix Sort چقدر است؟ پاسخ دقیق $\Theta(d(n+k))$ در همین پیاده‌سازی است. عبارت «سریع و پایدار» در اسلاید زمانی درست است که مرتب‌سازی هر رقم پایدار باشد و d و k برای اندازهٔ مسئله مناسب باشند.

> **تکمیلی — عددهای ۳۲بیتی:** می‌توان هر عدد را مثلاً به چهار رقمِ ۸بیتی تقسیم کرد. در این صورت $d = 4$ و هر رقم یکی از ۲۵۶ مقدار است؛ هر گذر آرایهٔ شمارندهٔ کوچکی دارد. این مثال، پاسخ عملی پرسش اسلاید ۲۰ را تکمیل می‌کند؛ جزئیات نمایش عددهای علامت‌دار در اسلایدها نیامده‌اند.

بخش 4

۴. جمع‌بندی و تمرین

۴٫۱ تصویر کلی درس

از اسلاید ۲۵. کران $\Omega(n \log n)$ متعلق به الگوریتم‌هایی است که اطلاعات ترتیب را **فقط از مقایسه** می‌گیرند. Merge Sort در بدترین حالت به این کران می‌رسد. Counting Sort از دامنهٔ کوچک کلیدها و Radix Sort از نمایش چندرقمی کلیدها استفاده می‌کنند؛ در شرایط مناسب می‌توانند نسبت به n خطی باشند.

ویژگی	مقایسه‌ای، مثل Merge Sort	Counting Sort	Radix Sort با Counting Sort
اطلاعات ترتیب	مقایسهٔ دو کلید	مقدار صحیح کلید	رقم‌های کلید
زمان	$\Theta(n \log n)$ برای Merge Sort	$\Theta(n + k)$	$\Theta(d(n + k))$
شرط مهم	امکان مقایسه	دامنهٔ $0..k$ و حافظهٔ کافی	نمایش d-رقمی و گذر پایدار
پایداری	به روش بستگی دارد	بله، با پیمایش معکوس	بله، اگر هر گذر پایدار باشد

چک‌لیست انتخاب روش. پیش از آنکه بگویی «این روش خطی است»، بپرس: k نسبت به n چقدر است؟ d ثابت است؟ آیا باید ترتیب رکوردهای هم‌کلید حفظ شود؟ آیا حافظهٔ $n + k$ در دسترس است؟

۴٫۲ پرسش‌های خود اسلایدها

۱. اسلاید ۵: بهترین زمان مرتب‌سازی مقایسه‌ای چیست؟ در بدترین حالت حداقل $\Omega(n \log n)$ و با Merge Sort حداکثر $O(n \log n)$ است؛ بنابراین مرتبهٔ بهینه $\Theta(n \log n)$ است. عبارت «حداقل n مقایسه» را به صورت کران خطیِ تقریبی بخوان؛ شکل دقیق آن در فصل ۱ آمده است.

۲. اسلاید ۱۷: زمان Counting Sort چیست؟ $\Theta(n + k)$، نه همیشه $\Theta(n)$.

۳. اسلاید ۲۰: چرا Counting Sort مستقیم برای همهٔ عددهای ۳۲بیتی مناسب نیست؟ دامنهٔ کلیدها ممکن است 2^{32} مقدار داشته باشد؛ شمارندهٔ مستقیم بسیار بزرگ می‌شود.

۴. اسلاید ۲۴: زمان Radix Sort چیست؟ با Counting Sort پایدار در هر گذر $\Theta(d(n + k))$ است.

۴٫۳ تمرین‌های تکمیلی

این تمرین‌ها در اسلایدها نیستند و برای سنجش فهم افزوده شده‌اند. ابتدا خودت حل کن؛ پاسخ‌های تشریحی در بخش بعد آمده است.

1. برای چهار کلید متمایز، درخت تصمیم حداقل چند مقایسه در بدترین حالت لازم دارد؟ آیا این کران به‌تنهایی الگوریتمی با همان تعداد مقایسه به ما می‌دهد؟
2. روی ورودی $A = [3, 1, 3, 0, 1]$ با $k = 3$، آرایهٔ شمارش و سپس آرایهٔ تجمعی C را بنویس. از راست به چپ، جای هر عنصر را در B تعیین کن.

3. دو رکورد «۲-آبی» و «۲-قرمز» با کلید مساوی ۲، به همین ترتیب در ورودی‌اند. اگر مرحلهٔ نهایی Counting Sort را از چپ به راست اجرا کنی، چه اتفاقی می‌افتد؟
4. ورودی $[21, 12, 11, 22]$ را با Radix Sort ده‌دهی دو گذره مرتب کن؛ آرایه را پس از یکان و پس از دهگان بنویس.
5. برای $n = 100000$، کدام گزینهٔ Counting Sort عملی‌تر است: کلیدهای $0..255$ یا $0..2^{32}-1$؟ دلیل را با زمان و حافظه توضیح بده.
6. جملهٔ «Quicksort بهینه است» را طوری بازنویسی کن که با بدترین حالت $O(n^2)$ آن تناقض نداشته باشد.

۴٫۴ پاسخ تشریحی تمرین‌ها

۱. تعداد ترتیب‌ها $4! = 24$ است، پس عمق درخت حداقل $\lceil \log_2 24 \rceil = 5$ است. این یک **کران پایین** است؛ صرفاً از آن نمی‌فهمیم درختی با پنج مقایسه برای همهٔ ورودی‌ها وجود دارد یا چگونه ساخته می‌شود.

۲. شمارش برای کلیدهای ۰ تا ۳ برابر $[1, 2, 0, 2]$ و تجمعی برابر $[1, 3, 3, 5]$ است. از راست به چپ: $A[5] = 1 \to B[3]$، $A[4] = 0 \to B[1]$، $A[3] = 3 \to B[5]$، $A[2] = 1 \to B[2]$، $A[1] = 3 \to B[4]$. در پایان $B = [0, 1, 1, 3, 3]$.

۳. اگر از انتهای بازهٔ کلید ۲ شروع کنیم ولی ورودی را از چپ بخوانیم، آبی در جای بزرگ‌تر و قرمز در جای کوچک‌تر قرار می‌گیرند؛ خروجیِ هم‌کلیدها قرمز، آبی می‌شود. بنابراین این ترکیب از جهت پیمایش و شمارنده پایدار نیست.

۴. پس از یکانِ پایدار: $[21, 11, 12, 22]$؛ زیرا یکان‌های ۱ پیش از ۲ می‌آیند و ترتیب هر گروه حفظ می‌شود. پس از دهگان: $[11, 12, 21, 22]$. اگر گذر دوم ناپایدار باشد، ممکن است ترتیب 11 و 12 خراب شود.

۵. بازهٔ ۰ تا ۲۵۵ فقط ۲۵۶ شمارنده می‌خواهد و $k \ll n$ است. بازهٔ ۳۲بیتی ممکن است حدود 2^{32} شمارنده بخواهد؛ زمان مقداردهی و حافظهٔ آن نامتناسب با $n = 100000$ است.

۶. «Quicksort تصادفی از نظر زمان موردانتظار $O(n \log n)$ هم‌مرتبه با کران پایین مرتب‌سازی مقایسه‌ای است، اما بدترین حالتش $O(n^2)$ است.»

۴٫۵ برگهٔ مرور یک‌دقیقه‌ای

- مقایسهٔ دودویی $\Rightarrow$ درخت تصمیم $\Rightarrow$ حداقل $n!$ برگ $\Rightarrow$ عمق $\Omega(n \log n)$.
- Counting Sort: **بشمار، تجمعی کن، از راست به چپ جای‌گذاری کن**؛ زمان $\Theta(n + k)$، حافظه $\Theta(n + k)$، پایدار.
- Radix Sort نوع LSD: از کم‌ارزش‌ترین رقم شروع کن و هر بار **پایدار** مرتب کن؛ زمان $\Theta(d(n + k))$.
- «خطی» همیشه مشروط به اندازهٔ دامنهٔ کلید، تعداد رقم‌ها و مدل ورودی است.